

LINUX SHELL SCRIPTING

/*BASICS*/

1. Write a script that declares two variables, adds them together, and prints the result. Explain how variable assignment works in Bash.

PROGRAM:

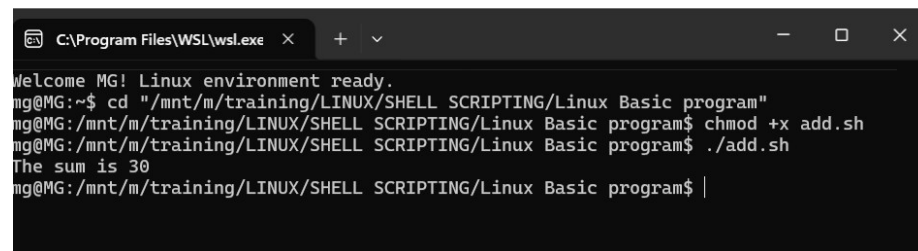
```
#!/bin/bash

a=10
b=20

sum=$((a+b))

echo "The sum is $sum"
```

OUTPUT:



```
C:\Program Files\WSL\wsl.exe x + v
Welcome MG! Linux environment ready.
mg@MG:~$ cd "/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program"
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x add.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./add.sh
The sum is 30
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

2. WAP to find the perimeter of a rectangle if length and breadth are given.

PROGRAM:

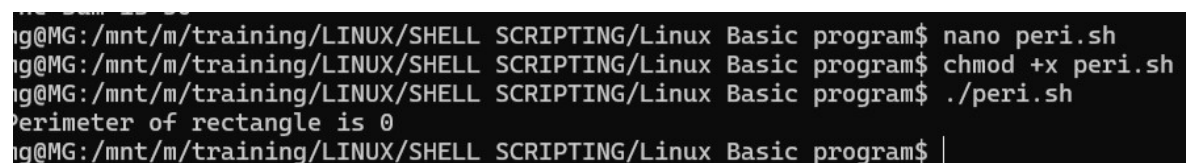
```
#!/bin/bash

length=$1
breadth=$2

perimeter=$((2*(length+breadth)))

echo "Perimeter of rectangle is $perimeter"
```

OUTPUT:



```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano peri.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x peri.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./peri.sh
Perimeter of rectangle is 60
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

/*IF*/

1. WAP to check whether the product of two given numbers is odd or even

PROGRAM:

```
#!/bin/bash
```

```
echo "Enter first number:"
```

```
read a
```

```
echo "Enter second number:"
```

```
read b
```

```
product=$((a*b))
```

```
if [ $((product % 2)) -eq 0 ]
```

```
then
```

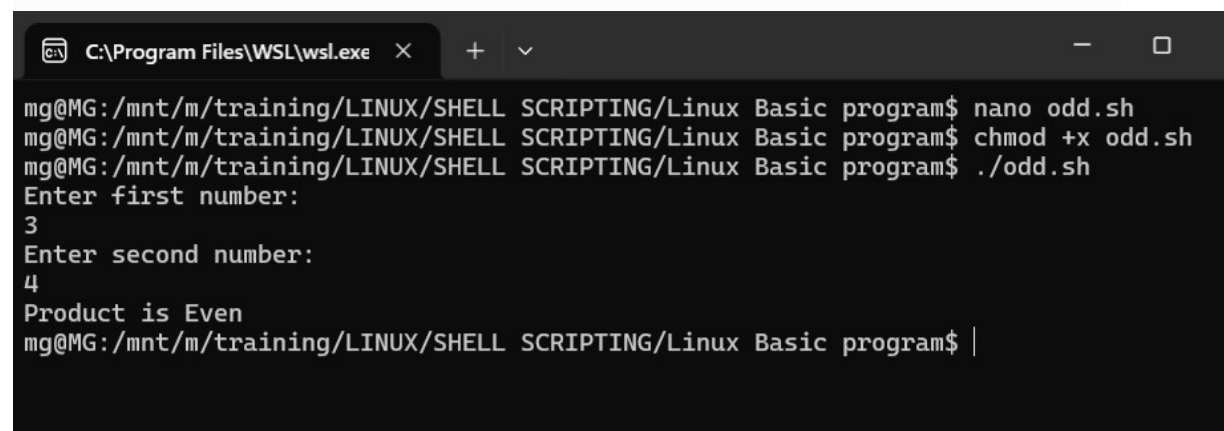
```
    echo "Product is Even"
```

```
else
```

```
    echo "Product is Odd"
```

```
Fi
```

OUTPUT:



```
C:\Program Files\WSL\wsl.exe
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano odd.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x odd.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./odd.sh
Enter first number:
3
Enter second number:
4
Product is Even
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

2. WAP to find the perimeter of a rectangle if length and breadth are given.

PROGRAM:

```
#!/bin/bash

echo "Enter length:"
read length

echo "Enter breadth:"
read breadth

perimeter=$((2*(length+breadth)))

echo "Perimeter of rectangle is $perimeter"
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano rect.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x rect.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./rect.sh
Enter length:
12
Enter breadth:
45
Perimeter of rectangle is 114
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

3. WAP, given three sides of a triangle, to check whether the triangle is perfect right triangle.

PROGRAM:

```
#!/bin/bash

echo "Enter three sides:"
read a b c

if [ $((a*a + b*b)) -eq $((c*c)) ] || \
    [ $((a*a + c*c)) -eq $((b*b)) ] || \
    [ $((b*b + c*c)) -eq $((a*a)) ]
then
    echo "It is a Right Triangle"
```

```
else
```

```
    echo "It is NOT a Right Triangle"
```

```
fi
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano tri.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x tri.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./tri.sh
Enter three sides:
4 7 2
It is NOT a Right Triangle
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

4. WAP to check whether given year is leap year or not.

PROGRAM:

```
#!/bin/bash
```

```
echo "Enter a year:"
```

```
read year
```

```
if [ $((year % 400)) -eq 0 ] || { [ $((year % 4)) -eq 0 ] && [ $((year % 100)) -ne 0 ]; }
```

```
then
```

```
    echo "Leap Year"
```

```
else
```

```
    echo "Not a Leap Year"
```

```
fi
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano leap.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x leap.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./leap.sh
./leap.sh: line 1: y#!/bin/bash: No such file or directory
Enter a year:
2000
Leap Year
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./leap.sh
./leap.sh: line 1: y#!/bin/bash: No such file or directory
Enter a year:
1900
Not a Leap Year
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

5. WAP that reads in 2 integers and determines and prints if the first is a multiple of the second.

PROGRAM:

```
#!/bin/bash

echo "Enter first integer:"
read a

echo "Enter second integer:"
read b

if [  $((a \% b)) -eq 0$  ]
then
    echo "$a is a multiple of $b"
else
    echo "$a is NOT a multiple of $b"
fi
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano multi.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x multi.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./multi.sh
Enter first integer:
12
Enter second integer:
3
12 is a multiple of 3
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

/*FOR, WHILE */

1. WAP to find the smallest and largest of entered n numbers.

PROGRAM:

```
#!/bin/bash

echo "Enter how many numbers:"
read n

echo "Enter number:"
read num
small=$num
large=$num

for ((i=2;i<=n;i++))
do
    echo "Enter number:"
    read num

    if [ $num -lt $small ]
    then
        small=$num
    fi

    if [ $num -gt $large ]
    then
        large=$num
    fi
done

echo "Smallest number = $small"
```

```
echo "Largest number = $large"
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano minmax.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x minmax.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./minmax.sh
Enter how many numbers:
5
Enter number:
5
Enter number:
7
Enter number:

./minmax.sh: line 16: [: -lt: unary operator expected
./minmax.sh: line 21: [: -gt: unary operator expected
Enter number:
3
Enter number:
7
Smallest number = 3
Largest number = 7
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

2. WAP to print all the even numbers from 1 to 100.

PROGRAM:

```
#!/bin/bash
```

```
for ((i=1;i<=100;i++))
```

```
do
```

```
    if [  $((i\%2))$  -eq 0 ]
```

```
    then
```

```
        echo $i
```

```
    fi
```

```
done
```

OUTPUT:

```
C:\Program Files\WSL\wsl.exe  X  +  v
mg@MG: /mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano even.sh
mg@MG: /mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x even.sh
mg@MG: /mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./even.sh
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
52
54
56
58
60
62
64
66
68
70
72
74
76
78
80
82
84
86
88
90
92
94
96
98
100
```

3. WAP to check whether a given number is prime or not.

PROGRAM:

```
#!/bin/bash
```

```
echo "Enter a number:"
```

```
read n
```

```
flag=0
```

```
for ((i=2;i<n;i++))
```

```

do
    if [  $((n\%i)) \neq 0$  ]
    then
        flag=1
        break
    fi
done

if [ $flag -eq 0 ]
then
    echo "Prime number"
else
    echo "Not a prime number"
fi

```

OUTPUT:

```

100
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano even.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano prime.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x prime.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./prime.sh
Enter a number:
53
Prime number
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |

```

4. WAP to find the sum and average of n numbers entered.

PROGRAM:

```

#!/bin/bash

echo "Enter number of values:"
read n

sum=0

for ((i=1;i<=n;i++))

```

```

do
    echo "Enter number:"
    read num
    sum=$((sum+num))
done

avg=$((sum/n))

echo "Sum = $sum"
echo "Average = $avg"

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano avg.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x avg.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano avg.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./avg.sh
Enter number of values:
4
Enter number:
2
Enter number:
8
Enter number:
6
Enter number:
4
Sum = 20
Average = 5
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |

```

5. WAP to print the numbers from m to n.

PROGRAM:

```

#!/bin/bash

echo "Enter starting number (m):"
read m

echo "Enter ending number (n):"
read n

```

```
for ((i=m;i<=n;i++))  
  
do  
  
    echo $i  
  
done
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano primeno.sh  
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano primeno.sh  
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x primeno.sh  
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./primeno.sh  
Enter starting number (m):  
7  
Enter ending number (n):  
20  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20
```

/*CASE STATEMENT*/

1. Write a simple script using case that prints the day of the week based on user input (e.g., 1 = Monday, 2 = Tuesday, etc.).

PROGRAM:

```
#!/bin/bash
```

```
echo "Enter a number (1-7):"
```

```
read day
```

```
case $day in
```

```
1) echo "Monday" ;;
```

```
2) echo "Tuesday" ;;
```

```
3) echo "Wednesday" ;;
```

```
4) echo "Thursday" ;;
```

```
5) echo "Friday" ;;
```

```
6) echo "Saturday" ;;  
7) echo "Sunday" ;;  
*) echo "Invalid input" ;;  
  
esac
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano days.sh  
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x days.s  
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./days.sh  
Enter a number (1-7):  
4  
Thursday  
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

2. Write a script that takes two numbers and an operator (+, -, *, /) as input. Use a case statement to perform the corresponding arithmetic operation.

PROGRAM:

```
#!/bin/bash  
  
echo "Enter first number:"  
read a  
  
echo "Enter second number:"  
read b  
  
echo "Enter operator (+ - * /):"  
read op  
  
case $op in  
+ ) result=$((a+b))  
    echo "Result = $result" ;;  
  
- ) result=$((a-b))  
    echo "Result = $result" ;;
```

```
\*) result=$((a*b))

    echo "Result = $result" ;;

/) result=$((a/b))

    echo "Result = $result" ;;

*) echo "Invalid operator" ;;

esac
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano op.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x op.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./op.sh
Enter first number:
3
Enter second number:
6
Enter operator (+ - * /):
-
Result = -3
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```

Traffic Light Simulation

Input can be red, yellow, or green.

Use a case statement to print:

Red → Stop

Yellow → Get Ready

Green → Go

PROGRAM:

```
#!/bin/bash

echo "Enter traffic light color (red/yellow/green):"
read color

case $color in
red)
    echo "Stop"
    ;;
```

```
yellow)
    echo "Get Ready"
;;

green)
    echo "Go"
;;

*)
    echo "Invalid input"
;;
esac
```

OUTPUT:

```
nano:sh: command not found
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ nano traffic.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ chmod +x traffic.sh
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ ./traffic.sh
Enter traffic light color (red/yellow/green):
yellow
Get Ready
mg@MG:/mnt/m/training/LINUX/SHELL SCRIPTING/Linux Basic program$ |
```